# 第 3 章：变量与运算符

本章目标：理解“名字绑定到对象”，避免把 Java 的变量槽位模型原样套到 Python；同时掌握常用运算符、短路求值、增强赋值和解包。配套代码：`example.py` / `Chapter03VariablesOperators.java`。

## 学习目标

学完本章后，你应该能够：

- 用“名字 → 对象”解释赋值、重新赋值和动态类型；
- 判断两个名字是“值相等”还是“指向同一个对象”；
- 预测可变对象在别名、浅复制和增强赋值下的变化；
- 正确使用算术、比较、逻辑、成员和位运算符；
- 使用交换、扩展解包和赋值表达式，并知道何时不该用它们。

## 3.1 一切皆对象：变量保存的是名字，不是对象本身

在 Python 中，整数、字符串、函数、类和模块都是对象。对象拥有类型和值；变量名只是程序用来找到对象的标签。执行赋值时，Python 先得到右侧对象，再让左侧名字绑定到它：

```
score = 95
alias = score
```

此时 `score` 和 `alias` 指向同一个整数对象。`type(score)` 给出对象的类型，`id(score)` 给出该对象在本次进程中的标识。`id` 适合调试身份关系，但不要持久化，也不要依赖它恰好表现得像内存地址。

| 要观察什么 | Python 写法 | 含义 |
| --- | --- | --- |
| 对象类型 | `type(value)` | 对象当前是什么类型 |
| 值是否相等 | `left == right` | 两个对象表达的值是否相等 |
| 是否同一对象 | `left is right` | 两个名字是否指向同一个对象 |

"一切皆对象"不等于"所有对象都能修改"。`int`、`str`、`tuple` 是不可变对象，不能原地改变；`list`、`dict`、`set` 是可变对象，可以在身份不变的情况下改变内容。

## 3.2 合法名字与命名约定

名字可包含字母、数字和下划线，但不能以数字开头，也不能使用 `if`、`for`、`class`、`def` 等关键字。`keyword.kwlist` 可以查看当前 Python 版本的完整关键字列表。.

Python 常见命名约定如下：

| 用途 | 约定 | 示例 |
| --- | --- | --- |
| 变量、函数 | `snake_case` | `order_total`、`calculate_total` |
| 类 | `PascalCase` | `OrderService` |
| 常量 | `UPPER_CASE` | `MAX_RETRY_COUNT` |
| 与关键字冲突 | 末尾加下划线 | `class_` |

`list`、`str`、`sum` 虽然不是关键字，却是内置名称。把它们用作普通变量会遮蔽内置对象：

```
# 教学反例：之后再调用 list(...) 会失败，因为 list 已经绑定到了一个字符串。
# list = "Python"
# values = list(range(3))

course_name = "Python"  # 使用能表达业务含义的名字。
```

## 3.3 动态类型与类型提示

Java 变量有编译期声明类型；Python 的类型属于对象，名字可以在运行时重新绑定到不同类型的对象：

```
result = 3
result = "three"  # 运行时允许，但频繁改变含义会降低可读性。
```

类型提示为读者、IDE 和静态检查器表达设计意图：

```
count: int = 3
```

类型提示为读者、IDE 和静态检查器表达设计意图：

```
count: int = 3
# count = "three"  # CPython 仍能执行；静态检查器应给出警告。
```

类型提示不是 Java 那样的运行时强制类型契约。实际项目仍应结合输入校验、测试和静态检查。即使 Python 允许重新绑定，也应让一个名字在一段代码中保持稳定含义。

名字必须先绑定才能读取。读取从未赋值的名字通常得到 `NameError`；如果函数内部既读取又给同名局部变量赋值，却在赋值前读取，则得到更具体的 `UnboundLocalError`。这不是 `null`，而是“当前作用域里还没有这个名字”。

## 3.4 赋值、别名与复制

普通赋值不会复制对象。对于可变对象，两个名字指向同一对象时，通过任一名字进行原地修改，另一名字也能看到：

```
original = ["Java", "Python"]
alias = original
alias.append("Go")

print(original)           # ['Java', 'Python', 'Go']
print(original is alias)  # True
```

列表的 `copy()` 或切片 `[:]` 会创建一个新的外层列表，但这只是**浅复制**：

```
nested = [["Java"], ["Python"]]
shallow = nested.copy()

nested.append(["Go"])       # 外层独立，shallow 看不到新元素。
nested[0].append("Kotlin")   # 内层共享，shallow 也能看到 Kotlin。
```

只有确实需要整棵嵌套结构相互独立时才考虑 `copy.deepcopy`。业务对象可能包含文件、连接或自定义复制规则，深复制并不总是合理；更好的选择有时是重新构造明确需要的数据。

对于不可变对象，“修改”实际会产生新对象并重新绑定名字：

```
total = 10
```

```
total = 10
old_total = total
total += 5

print(total)          # 15
print(old_total)      # 10
print(total is old_total)  # False
```

## 3.5 算术运算符

| 运算符 | 含义 | 示例结果 |
| --- | --- | --- |
| `+`、`-`、`*` | 加、减、乘 | `7 + 2 == 9` |
| `/` | 真除法，总是得到浮点数 | `7 / 2 == 3.5` |
| `//` | 向负无穷取整的整除 | `-7 // 2 == -4` |
| `%` | 余数 | `17 % 5 == 2` |
| `**` | 幂 | `2 ** 8 == 256` |
| `divmod(a, b)` | 同时得到整除结果和余数 | `divmod(17, 5) == (3, 2)` |

Java 整数除法向 0 截断，Python 的 `//` 向负无穷取整，因此负数结果可能不同。除数为 0 时，`/`、`//` 和 `%` 都会抛出 `ZeroDivisionError`。

运算优先级大体遵循“括号、幂、正负号、乘除、加减、比较、逻辑”。当读者需要停下来回忆优先级时，直接加括号通常更清楚：

```
subtotal = 100
shipping = 12
discount_rate = 0.1
payable = (subtotal + shipping) * (1 - discount_rate)
```

一个容易忽略的边界是 `-2 ** 2 == -4`，因为它相当于 `-(2 ** 2)`；若底数是负数，应明确写成 `(-2) ** 2 == 4`。

## 3.6 比较、成员与身份运算符

## 3.6 比较、成员与身份运算符

比较运算符是 `== != < <= > >=`。Python 支持链式比较：

```
age = 30
is_working_age = 18 <= age < 65
```

它表达的是 `18 <= age and age < 65`，但 `age` 只求值一次。Java 不支持这种语法，需要用 `&&` 连接两个比较。

`in / not in` 用于成员判断；对列表检查元素，对字符串检查子串，对字典默认检查键：

```
roles = ["editor", "reviewer"]
can_review = "reviewer" in roles
```

`==` 比较值，`is` 比较对象身份。不要对业务字符串、整数或列表内容使用 `is`；CPython 可能复用某些小整数和字符串对象，但这是实现细节，不是值比较规则。`None` 是单例，缺失值判断应写成 `value is None` 或 `value is not None`。

## 3.7 真假值、逻辑运算与短路求值

`None`、数值零、空字符串和空容器是假值，其余对象通常为真。`and`、`or` 会**短路求值**：只要左侧已经决定结果，就不会计算右侧。这可以安全地把检查放在危险操作之前：

```
denominator = 0
is_large = denominator != 0 and 100 / denominator > 10
```

上例结果是 `False`，除法没有执行。顺序反过来则会先触发除零错误。

与 Java 的 `&&`、`||` 不同，Python 的 `and` 和 `or` 返回参与运算的某个对象，不保证返回 `bool`：

```
nickname = ""
display_name = nickname or "匿名用户"  # 得到字符串"匿名用户"
```

这适合"所有假值都表示缺失"的简单默认值，但如果 `0`、`False` 或空容器是有效业务值，

README README README
z72502x6dg8fpjj0sff40000gp/T/crossnote-f-9475ef2ee5f29880b72c1ecd0c9.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家

这适合“所有假值都表示缺失”的简单默认值，但如果 `0` 、 `False` 或空容器是有效业务值，`or` 会误伤它们。只想处理缺失值时，应显式判断 `is None`：

```
configured_timeout = 0
timeout = 30 if configured_timeout is None else configured_timeout
```

`not value` 一定返回真正的布尔值。若接口必须返回 `bool`，可使用 `bool(expression)` 明确转换。

## 3.8 增强赋值：`+=` 不总等于 `x = x + y`

`+=` 、 `-=` 、 `*=` 等称为增强赋值。它对可变对象和不可变对象的可观察效果不同：

```
numbers = [1, 2]
alias = numbers
numbers += [3]       # 列表通常原地扩展。
print(alias)         # [1, 2, 3]

numbers = numbers + [4]  # + 创建新列表，然后 numbers 重新绑定。
print(alias)             # 仍是 [1, 2, 3]
```

对整数执行 `+=` 时，因为整数不可变，会计算新整数并重新绑定。对列表执行 `+=` 时，列表实现通常原地修改，因此所有别名都能看到变化。可变对象参数尤其要留意这一差异：

```
def append_in_place(values: list[int]) -> None:
    values += [99]  # 调用方能看到。


def append_by_rebinding(values: list[int]) -> None:
    values = values + [100]  # 只改变局部名字，调用方看不到。
```

## 3.9 多目标赋值、解包与赋值表达式

Python 会先计算赋值号右侧，再把结果绑定到左侧，因此交换两个名字不需要临时变量：

```
left, right = right, left
```

README README README
502x6dg8fpjj0sff40000gp/T/crossnote-f-9475ef2ee5f29880b72c1ecd0c9.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下

## 3.9 多目标赋值、解包与赋值表达式

Python 会先计算赋值号右侧，再把结果绑定到左侧，因此交换两个名字不需要临时变量：

```
left, right = right, left
```

解包要求元素数量匹配；扩展解包中的一个 `*name` 可以接收剩余元素：

```
first, *middle, last = [10, 20, 30, 40]
# first == 10, middle == [20, 30], last == 40
```

数量不匹配会抛出 `ValueError`。即使不用某一项，也要明确接收它；按约定可用 `_` 表示“有意忽略”：`name, _, city = profile`。

链式赋值对不可变值通常直观，但对可变对象会制造别名：

```
# 教学反例：两个名字指向同一个列表。
first_items = second_items = []

# 需要两个独立列表时分别创建。
first_items = []
second_items = []
```

Python 3.8+ 的赋值表达式 `:=`（常称“海象运算符”）能在表达式内部绑定名字：

```
course_name = "Python"
if (name_length := len(course_name)) >= 6:
    print(name_length)
```

外层括号能让意图更明确。只有它确实避免重复计算并保持可读性时才使用；普通赋值加一个清晰的 `if` 往往更适合初学者和复杂业务逻辑。

## 3.10 位运算符

位运算处理整数的二进制位，常见于权限掩码、协议字段和底层标志：

| 运算符 | 含义 | 示例 |
| --- | --- | --- |

## 3.10 位运算符

位运算处理整数的二进制位，常见于权限掩码、协议字段和底层标志：

| 运算符 | 含义 | 示例 |
| --- | --- | --- |
| `&` | 按位与 | 检查某个标志位 |
| `\|` | 按位或 | 合并标志位 |
| `^` | 按位异或 | 不同的位保留为 1 |
| `~` | 按位取反 | 翻转所有位 |
| `<<` 、`>>` | 左移、右移 | 按位移动 |

```
READ = 0b001
WRITE = 0b010
permissions = READ | WRITE
can_write = bool(permissions & WRITE)
```

不要把 `&` / `|` 当作 `and` / `or` 的替代品。前者进行位运算且不会按同样规则短路，后者表达逻辑选择。

## 常见错误速查

| 错误写法或现象 | 原因 | 推荐做法 |
| --- | --- | --- |
| `text is "ok"` | 把身份当作值比较 | 写 `text == "ok"` |
| `timeout = value or 30` 丢失 `0` | `0` 是假值 | 只处理缺失时判断 `is None` |
| `left = right = []` 后互相影响 | 两个名字共享一个列表 | 分别创建两个列表 |
| 以为 `copied = nested.copy()` 完全独立 | 浅复制仍共享内层对象 | 重建数据或谨慎使用深复制 |
| 以为类型提示会阻止错误赋值 | CPython 默认不强制提示 | 配合静态检查、校验和测试 |
| `values += extra` 意外修改调用方数据 | 可变对象可能原地修改 | 明确是否允许修改，必要时先复制 |

## Java 心智切换

## 常见错误速查

| 错误写法或现象 | 原因 | 推荐做法 |
| --- | --- | --- |
| `text is "ok"` | 把身份当作值比较 | 写 `text == "ok"` |
| `timeout = value or 30` 丢失 `0` | `0` 是假值 | 只处理缺失时判断 `is None` |
| `left = right = []` 后互相影响 | 两个名字共享一个列表 | 分别创建两个列表 |
| 以为 `copied = nested.copy()` 完全独立 | 浅复制仍共享内层对象 | 重建数据或谨慎使用深复制 |
| 以为类型提示会阻止错误赋值 | CPython 默认不强制提示 | 配合静态检查、校验和测试 |
| `values += extra` 意外修改调用方数据 | 可变对象可能原地修改 | 明确是否允许修改，必要时先复制 |

## Java 心智切换

把 Python 赋值读作“让名字指向对象”，而不是“把值装进有固定类型的盒子”。每当代码出现别名、函数参数、`+=` 或嵌套容器时，问自己三个问题：对象是否可变、这一步产生新对象还是原地修改、还有哪些名字指向它。这个模型能统一解释动态类型、共享修改和局部重新绑定。

Python 的运算符也更偏向直接表达意图：链式比较、成员判断和解包通常比手写索引更清楚；但 `and / or` 返回对象、`//` 的负数语义以及 `is` 的身份判断，都不能机械套用 Java 经验。

## 小练习

1. 先不运行代码，预测 `example.py` 中“别名与浅复制”和“增强赋值”两节的每一行输出，再运行并解释与你预测不同的地方。
2. 写一个 `normalize_page_size(value)` 函数：`None` 使用默认值 20，但合法值 `0` 必须保留。不要使用会误伤 `0` 的 `value or 20`。
3. 给定 `permissions = 0b101`，分别判断读权限 `0b001`、写权限 `0b010` 和管理权限 `0b100` 是否开启，再用按位或添加写权限。
4. 写一个函数接收列表：先演示 `values += [99]`，再演示 `values = values + [100]`。记录函数内外的值与身份关系，解释调用者为什么只能看到其中一次变化。
5. 给定 `["Java", "Python", "Go", "Rust"]`，用扩展解包取得第一项、最后一项和中间项；再故意让普通解包的变量数量不匹配，阅读并解释 `ValueError`，最后把错误代码注释掉。

```python
"""第 3 章：名字绑定、对象别名、运算符、增强赋值和解包。"""

import keyword


def print_section(title: str) -> None:
    """打印统一的小节标题，让运行结果能和 README 对照。"""
    print(f"\n=== {title} ===")


def normalize_page_size(value: int | None) -> int:
    """只把缺失值替换为 20，合法的 0 必须原样保留。"""
    return 20 if value is None else value


def show_names_and_types() -> None:
    """观察名字如何绑定对象，以及类型提示和运行时类型的区别。"""
    print_section("1. 名字绑定、类型与关键字")

    print("部分 Python 关键字:", ", ".join(keyword.kwlist[:]))

    score: int = 95
    score_alias: Literal[95] = score
    print("score 的值/类型:", score, type(score).__name__)
    print("直接赋值后指向同一对象:", score is score_alias)

    # 类型提示描述设计意图，但 CPython 不会像 Java 编译器那样强制它。
    # 教学反例（保持注释）：把 score 改成 "优秀" 可以运行，
    # 却会被静态检查器警告。
    print("推荐让 score 始终表达整数分数，避免随意重新绑定为字符串")

    # 教学反例（保持注释）：不要遮蔽 list、str、sum 等内置名称。
    # list = "Python"
    # values = list(range(3))  # 此时会得到 TypeError: 'str' object is not callable
    course_name = "Python"
    print("有业务含义的名字:", course_name)

```

```python
def show_aliases_and_copies() -> None:
    """用修改前后结果展示别名、浅复制和嵌套对象共享。"""
    print_section("2. 别名与浅复制")

    original: list[str] = ["Java", "Python"]
    alias: list[str] = original
    copied: list[str] = original.copy()

    alias.append("Go")
    print("通过 alias 修改后 original:", original)
    print("外层已复制的 copied:", copied)
    print("original is alias:", original is alias)
    print("original is copied:", original is copied)

    nested: list[list[str]] = [["Java"], ["Python"]]
    shallow: list[list[str]] = nested.copy()
    nested.append(["Go"])
    nested[0].append("Kotlin")

    # 新增外层元素不会进入 shallow；但共享的第一个内层列表会同时变化。
    print("nested 新增了外层元素:", nested)
    print("shallow 仍共享内层元素:", shallow)
    print("外层是否相同:", nested is shallow)
    print("第一个内层列表是否相同:", nested[0] is shallow[0])


def show_arithmetic_and_comparisons() -> None:
    """展示最容易和 Java 产生语义差异的常用运算符。"""
    print_section("3. 算术、比较、成员与身份")

    quotient: int, remainder: int = divmod(17, 5)
    print("真除法 17 / 5:", 17 / 5)
    print("整除与余数:", 17 // 5, 17 % 5)
    print("divmod 同时返回:", quotient, remainder)
    print("负数整除 -7 // 2:", -7 // 2)
    print("幂 2 ** 8:", 2**8)

```

```python
def show_arithmetic_and_comparisons() -> None:

    age = 30
    print("链式比较 18 <= age < 65:", 18 <= age < 65)

    roles: list[str] = ["editor", "reviewer"]
    print("成员判断 'reviewer' in roles:", "reviewer" in roles)
    print("成员判断 'admin' not in roles:", "admin" not in roles)

    first: list[int] = [1, 2]
    second: list[int] = [1, 2]
    print("first == second (值相等):", first == second)
    print("first is second (不是同一对象):", first is second)

    missing = None
    print("None 使用身份判断:", missing is None)


def show_boolean_operators() -> None:
    """观察短路求值，以及 and/or 返回对象而非固定返回 bool。"""
    print_section("4. 真假值、逻辑运算与短路")

    denominator = 0
    # 左侧为 False 后，右侧除法不会执行，因此不会出现 ZeroDivisionError。
    is_large: Literal[False] = denominator != 0 and 100 / denominator > 10
    print("短路避免除零:", is_large)

    nickname: Literal[''] = ""
    display_name: Literal['匿名用户'] = nickname or "匿名用户"
    print("or 返回选中的对象:", display_name, type(display_name).__name__)

    # 0 是有效配置，不能用 configured_timeout or 30，否则会错误地得到 30。
    configured_timeout: int | None = 0
    timeout: Literal[30] | Literal[0] = 30 if configured_timeout is None else configured_timeout
    print("显式判断 None，保留有效的 0:", timeout)
    print("not 一定返回 bool:", not [], type(not []).__name__)

```

```
    """列表的 += 通常原地扩展，因此调用方能看到变化。"""
    before: list[int] = values
    values += [99]
    print("函数内 += 后仍是同一列表:", values is before)


def append_by_rebinding(values: list[int]) -> None:
    """列表的 + 创建新列表；这里只重新绑定了函数内的局部名字。"""
    before: list[int] = values
    values = values + [100]
    print("函数内 + 后是新列表:", values is not before)
    print("函数内局部值:", values)


def show_augmented_assignment() -> None:
    """对比不可变整数和可变列表执行增强赋值时的身份变化。"""
    print_section("5. 增强赋值与重新绑定")

    points = 10
    old_points: Literal[10] = points
    points += 5
    print("整数 += 后的新旧值:", points, old_points)
    print("整数 += 后是否仍是同一对象:", points is old_points)

    numbers: list[int] = [1, 2]
    alias: list[int] = numbers
    append_in_place(numbers)
    print("调用方看到 += 的修改:", numbers)
    print("原别名也看到 += 的修改:", alias)

    append_by_rebinding(numbers)
    print("调用方看不到函数内 + 的局部重绑定:", numbers)


def show_unpacking_and_assignment_expression() -> None:
```

```python

def show_unpacking_and_assignment_expression() -> None:
    """展示交换、普通解包、扩展解包和适度使用赋值表达式。"""
    print_section("6. 交换、解包与赋值表达式")

    left, right = "L", "R"
    left: Literal['R'], right: Literal['L'] = right, left
    print("先计算右侧再赋值，因此可直接交换:", left, right)

    username, _, city = ("alice", 30, "Shanghai")
    print("普通解包并忽略不需要的年龄:", username, city)

    first: int, *middle, last: int = [10, 20, 30, 40]
    print("扩展解包:", first, middle, last)

    # 教学反例（保持注释）：数量不匹配会中断脚本。
    # only_one, only_two = [1, 2, 3]

    course_name = "Python"
    if (name_length := len(course_name)) >= 6:
        print("赋值表达式避免重复计算长度:", name_length)


def show_bitwise_operators() -> None:
    """用权限掩码展示位运算的典型业务用途。"""
    print_section("7. 位运算：权限掩码")

    read_permission = 0b001
    write_permission = 0b010
    admin_permission = 0b100

    permissions: Literal[3] = read_permission | write_permission
    print("合并后的权限（二进制）:", f"{permissions:03b}")
    print("允许读取:", bool(permissions & read_permission))
    print("允许写入:", bool(permissions & write_permission))
    print("允许管理:", bool(permissions & admin_permission))

```

```python

def demonstrate_readme_exercises() -> None:
    """运行 README 中关于默认值、权限、身份与解包的练习。"""
    print_section("README 小练习")

    print("练习 1: 别名、浅复制和增强赋值的逐行结果见上方第 2、5 节")

    print("练习 2: 只为 None 提供分页默认值")
    for page_size in (None, 0, 50):
        print(f"  normalize_page_size({page_size!r}) -> {normalize_page_size(page_size)}")

    print("练习 3: 权限掩码")
    read_permission = 0b001
    write_permission = 0b010
    admin_permission = 0b100
    permissions = 0b101
    print("  初始权限:", f"{permissions:03b}")
    print("  读/写/管理:", *(bool(permissions & bit) for bit in (
        read_permission,
        write_permission,
        admin_permission,
    )))
    permissions |= write_permission
    print("  添加写权限后:", f"{permissions:03b}")

    print("练习 4: 函数内原地修改与局部重绑定")
    values: list[int] = [1, 2]
    caller_alias: list[int] = values
    append_in_place(values)
    print("  += 返回后调用方:", values, "身份不变:", values is caller_alias)
    append_by_rebinding(values)
    print("  + 返回后调用方:", values, "身份不变:", values is caller_alias)

    print("练习 5: 扩展解包与数量不匹配")
    first: str, *middle, last: str = ["Java", "Python", "Go", "Rust"]
    print("  第一项/中间项/最后一项:", first, middle, last)
    try:
```

```
184  def demonstrate_readme_exercises() -> None:
217      first: str, *middle, last: str = ["Java", "Python", "Go", "Rust"]
218      print("  第一项/中间项/最后一项:", first, middle, last)
219      try:
220          one: str, two: str = ["Java", "Python", "Go"]
221          print(one, two)  # 变量数量不匹配，因此不会执行到这里。
222      except ValueError as error:
223          print("  普通解包数量不匹配:", type(error).__name__, "-", error)
224
225
226  def main() -> None:
227      show_names_and_types()
228      show_aliases_and_copies()
229      show_arithmetic_and_comparisons()
230      show_boolean_operators()
231      show_augmented_assignment()
232      show_unpacking_and_assignment_expression()
233      show_bitwise_operators()
234      demonstrate_readme_exercises()
235
236
237  if __name__ == "__main__":
238      main()
239
```